

Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 1



Nama:

Muhamad Rizqi Candra

NIM:

20240801035

1. Pendahuluan

Dokumentasi ini berisi proses pengerjaan Praktikum Flutter Fundamental Pertemuan 1, mulai dari persiapan dan pembuatan project sampai implementasi aplikasi. Selain itu, dokumentasi ini juga menjelaskan hasil dari setiap tahap yang sudah dikerjakan.

2. Tujuan Praktikum:

- Menenal Flutter dan Dart.
- Melakukan instalasi dan konfigurasi Flutter.
- Membuat project Flutter pertama.
- Memahami struktur project Flutter.
- Memahami konsep dasar widget.
- Menggunakan hot reload.
- Membuat aplikasi sederhana menggunakan widget Flutter.

3. Tools yang digunakan

Komponen	Keterangan
OS	Windows 11
Framework	Flutter
Bahasa	Dart
IDE	Android Studio
Emulator	Android Emulator
SDK	Android SDK
Device	Emulator / HP android

4. Konsep Dasar

- Flutter : UI toolkit dari Google untuk membangun aplikasi mobile, web, dan desktop dari satu basis kode
- Dart : Bahasa pemrograman Flutter
- Widget: semua elemen di Flutter (teks, tombol, layout, bahkan aplikasi itu sendiri) adalah widget.
- StatelessWidget: widget yang tampilannya tidak berubah.
- StatefulWidget: widget yang tampilannya bisa berubah lewat setState().
- Hot Reload: perubahan kode langsung tampil tanpa restart aplikasi.

5. Langkah Praktikum

Setelah Flutter dan Android Studio selesai diinstal, saya melakukan pengecekan terlebih dahulu untuk memastikan Flutter SDK, Android Studio, dan emulator sudah dapat digunakan dengan baik.

Pengecekan dilakukan melalui perintah flutter doctor, kemudian project dijalankan menggunakan flutter run.

```
flutter doctor
```

```
flutter run
```

Setelah tidak ditemukan masalah pada environment, saya membuat project Flutter melalui Command Prompt menggunakan perintah flutter create. Setelah project berhasil dibuat, saya masuk ke folder project dan menjalankannya.

```
flutter create praktikum1
```

```
cd praktikum1
```

```
flutter run
```

Setelah project berhasil dibuat, project dibuka melalui Android Studio untuk melanjutkan proses implementasi. Secara umum, struktur project Flutter yang digunakan terdiri dari beberapa bagian berikut:

Folder	Fungsi
lib/main.dart	Titik masuk aplikasi (kode utama)
pubspec.yaml	Konfigurasi proyek dan aset
android/, ios/	Kode platform
test/	Berkas pengujian

6. Implementasi Hello Flutter

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 1',
      home: Scaffold(
        appBar: AppBar(title: const Text('Hello Flutter')),
        body: const Center(
          child: Text(
            'Halo, nama saya [NAMA]!',
            style: TextStyle(fontSize: 24),
          ),
        ),
      ),
    );
  }
}
```

Pada tahap ini saya membuat aplikasi sederhana dengan tampilan "Hello Flutter". Fungsi main() digunakan sebagai titik awal program dan menjalankan MyApp. Di dalam MyApp terdapat MaterialApp dan Scaffold. AppBar digunakan untuk menampilkan judul "Hello Flutter", sedangkan bagian body menampilkan teks "Halo, nama saya [NAMA]!" dengan ukuran font 24.

7. Implementasi Layout menggunakan Column

```
body: Center(  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: const [  
      Icon(Icons.flutter_dash, size: 80, color: Colors.blue),  
      SizedBox(height: 16),  
      Text('Halo, nama saya [NAMA]!', style: TextStyle(fontSize: 24)),  
      Text('NIM: [NIM]'),  
    ],  
  ),  
)
```

Pada bagian body, saya menggunakan Column untuk menyusun beberapa komponen secara vertikal dan menempatkannya di tengah halaman. Komponen yang ditampilkan terdiri dari ikon Flutter, teks sapaan "Halo, nama saya [NAMA]!" dengan ukuran font 24, serta teks NIM.

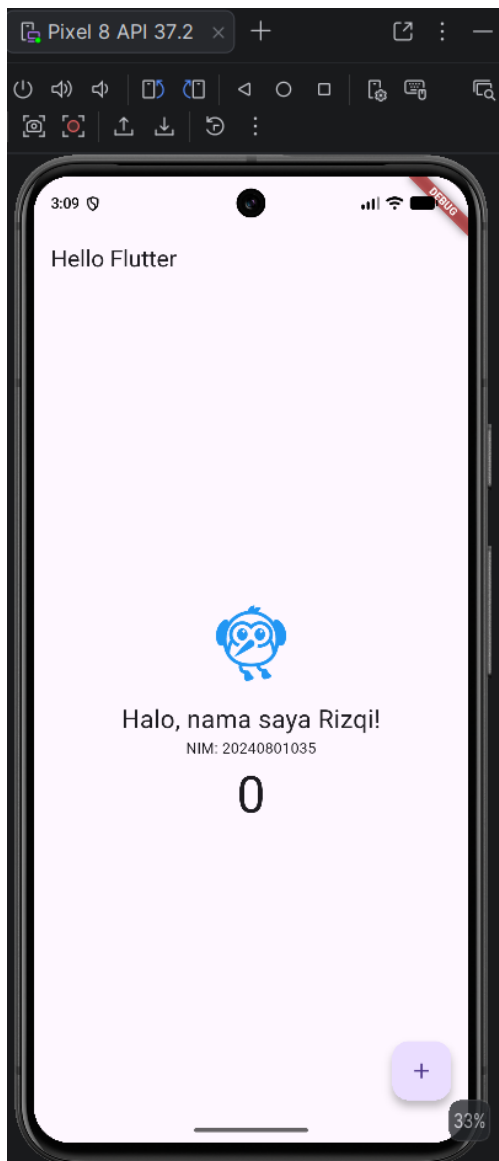
8. Implementasi StatefulWidget

Pada tahap berikutnya saya menggunakan StatefulWidget agar tampilan aplikasi dapat berubah ketika terjadi suatu aksi. StatefulWidget cocok digunakan pada bagian aplikasi yang memiliki data atau nilai yang dapat berubah. Saya menambahkan class CounterPage di bawah MyApp dan mengubah bagian home menjadi const CounterPage().

```
class CounterPage extends StatefulWidget {  
  const CounterPage({super.key});  
  
  @override  
  State<CounterPage> createState() => _CounterPageState();  
}  
  
class _CounterPageState extends State<CounterPage> {  
  int _count = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Counter Saya')),  
      body: Center(  
        child: Text('$_count', style: const TextStyle(fontSize: 48)),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: () => setState(() => _count++),  
        child: const Icon(Icons.add),  
      ),  
    );  
  }  
}
```

CounterPage merupakan halaman yang menyimpan nilai `_count`. Pada tampilannya terdapat AppBar dengan judul "Counter Saya", nilai angka di bagian tengah, serta tombol tambah (+) di bagian kanan bawah. Ketika tombol ditekan, `setState()` digunakan untuk menambah nilai `_count` sehingga angka pada layar ikut diperbarui.

Hasil Akhir:



Dari implementasi tersebut, ketika tombol + ditekan, nilai yang ditampilkan di tengah layar akan bertambah. Perubahan ini terjadi karena nilai `_count` diperbarui menggunakan `setState()`.

9. Latihan Mandiri

a) Latihan 1: Mengubah warna AppBar dan teks.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Colors.brown,
      title: const Text(
        'Hello Flutter',
        style: TextStyle(color: Colors.white),
      ),
    ),
  ),
),
AppBar

const Text(
  'Halo, nama saya Rizqi!',
  style: TextStyle(
    fontSize: 24,
    color: Colors.brown,
  ),
),
TextStyle
),
Text
const Text(
  'NIM: 20240801035',
  style: TextStyle(color: Colors.brown),
),
Text
Text(
  '$_count',
  style: const TextStyle(
    fontSize: 48,
    color: Colors.brown,
  ),
),
TextStyle
),
Text
```

Pada latihan pertama, saya mengubah warna tema aplikasi menjadi hijau. Perubahan diterapkan pada AppBar dan bagian teks nama serta NIM sehingga tampilan aplikasi menjadi lebih seragam.

b) Tambahkan tombol kedua (Icons.remove) untuk mengurangi angka.

```
floatingActionButton: Row(
  mainAxisAlignment: MainAxisAlignment.end,
  children: [
    FloatingActionButton(
      onPressed: () => setState(() => _count--),
      child: const Icon(Icons.remove),
    ),
  ],
),
```

Tombol minus ditempatkan di dalam Row dan diratakan ke bagian kanan menggunakan MainAxisAlignment.end. Ketika tombol ditekan, setState(() => _count--) dijalankan sehingga nilai _count berkurang satu. Setelah itu, tampilan aplikasi diperbarui dan angka yang ditampilkan ikut berubah. Pada tahap ini angka masih dapat menjadi negatif karena belum diberi batas minimum.

- c) Tambahkan tombol reset yang mengembalikan angka ke 0.

```
floatingActionButton: FloatingActionButton(  
  onPressed: () => setState(() => _count++),  
  child: const Icon(Icons.add),  
), FloatingActionButton
```

Tombol reset digunakan untuk mengembalikan nilai `_count` menjadi 0. Ketika tombol ditekan, `setState()` akan mengubah nilai tersebut menjadi 0 dan tampilan aplikasi langsung diperbarui. Ikon yang digunakan pada tombol berfungsi sebagai penanda visual untuk fitur reset.

- d) Cegah angka menjadi negatif.

```
floatingActionButton: Row(  
  mainAxisAlignment: MainAxisAlignment.end,  
  children: [  
    FloatingActionButton(  
      onPressed: () {  
        if (_count > 0) {  
          setState(() => _count--);  
        }  
      },  
      child: const Icon(Icons.remove),  
    ), FloatingActionButton  
  ],  
  const SizedBox(width: 16),
```

Pada latihan ini saya memberikan kondisi agar nilai `_count` tidak menjadi negatif. Jika nilai masih lebih dari 0, tombol minus dapat digunakan untuk mengurangi angka satu per satu. Namun, ketika nilai sudah 0, tombol minus dibuat tidak aktif sehingga nilai tidak dapat berkurang lagi.

Hasil akhir dari latihan 1 sampai 4 menunjukkan bahwa warna tampilan sudah berubah, tersedia tombol tambah, kurang, dan reset, serta nilai counter tidak dapat turun di bawah 0.



10. Tugas

Pada tugas ini saya membuat aplikasi satu halaman berupa "Kartu Perkenalan". Informasi yang ditampilkan terdiri dari ikon, nama, NIM, jurusan, dan hobi. Komponen yang digunakan antara lain Column, Text, Icon, dan SizedBox.

a) Penjelasan Implementasi

- **MaterialApp:**

```
return MaterialApp(  
  title: 'Kartu Perkenalan',  
  MaterialApp digunakan sebagai dasar aplikasi Flutter. Pada bagian ini saya  
  memberikan judul aplikasi yaitu "Kartu Perkenalan".
```
- **Scaffold dan AppBar:**

```
home: Scaffold(  
  appBar: AppBar(  
    title: const Text('Kartu Perkenalan'),  
  ),
```

Scaffold digunakan sebagai struktur utama halaman aplikasi, sedangkan AppBar digunakan untuk menampilkan bagian atas halaman dengan judul "Kartu Perkenalan".

- Column:

```
child: Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: const [
```

Column digunakan untuk menyusun seluruh informasi secara vertikal. Penggunaan MainAxisAlignment.center membuat isi Column berada di bagian tengah halaman.

- Icon:

```
Icon(  
  Icons.child_care,  
  size: 100,  
  color: Colors.brown,  
),
```

- SizedBox:

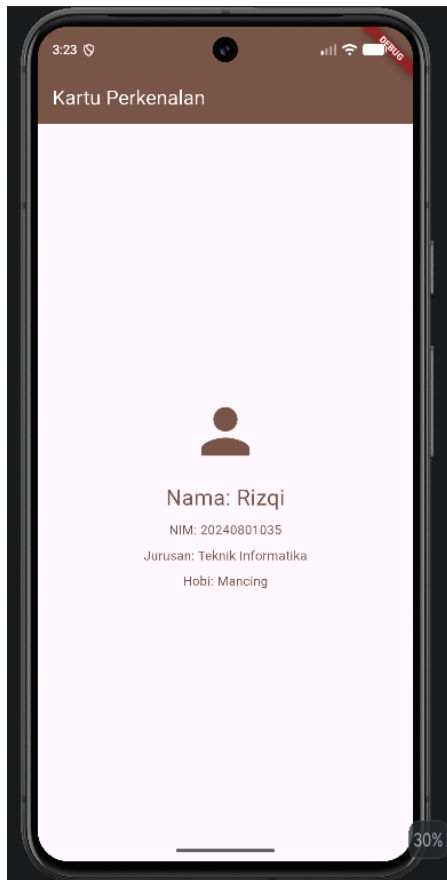
```
SizedBox(height: 16),
```

SizedBox digunakan untuk memberikan jarak antara ikon dan informasi yang ditampilkan di bawahnya.

- Text

```
Text(  
  'Muhamad Rizqi Candra',  
  style: TextStyle(fontSize: 25),  
),  
  
Text('NIM: 20240801035'),  
Text('Jurusan: Teknik Informatika'),  
Text('Hobi: Mancing'),
```

11. Hasil Akhir Tugas



Setelah seluruh tahap praktikum selesai dilakukan, aplikasi Flutter berhasil dibuat dan dijalankan menggunakan Android Emulator. Dari praktikum ini saya memahami struktur dasar project Flutter dan penggunaan beberapa widget seperti Text, Icon, Column, dan SizedBox. Saya juga memahami penggunaan StatefulWidget dan setState() untuk membuat tampilan yang dapat berubah sesuai dengan aksi pengguna.

12. Troubleshooting

Masalah	Solusi
flutter tidak dikenali	Periksa PATH, buka ulang terminal
Lisensi Android belum diterima	flutter doctor --android-licenses
Emulator lambat	Aktifkan virtualisasi (VT-x/AMD-V) di BIOS
Perangkat tidak terdeteksi	Aktifkan USB debugging, cek dengan flutter devices

13. Pertanyaan

- a) Apa perbedaan StatelessWidget dan StatefulWidget?
- b) Mengapa perubahan `_count` perlu menggunakan `setState()`?
- c) Apa keuntungan menggunakan hot reload?

Jawaban:

- a) StatelessWidget digunakan untuk widget yang tampilannya tidak bergantung pada perubahan state, sehingga tampilannya cenderung tetap. Sementara itu, StatefulWidget digunakan ketika terdapat data atau kondisi yang dapat berubah selama aplikasi berjalan.
- b) Perubahan nilai `_count` perlu menggunakan `setState()` karena Flutter perlu mengetahui bahwa terdapat perubahan state yang harus ditampilkan kembali. Dengan `setState()`, Flutter akan membangun ulang bagian tampilan yang berkaitan sehingga angka pada layar ikut berubah.
- c) Hot reload memudahkan proses pengembangan karena perubahan kode dapat langsung dilihat pada aplikasi tanpa harus menjalankan ulang aplikasi dari awal. Dengan begitu, proses mencoba dan memperbaiki tampilan menjadi lebih cepat.

14. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, saya mendapatkan pemahaman dasar mengenai penggunaan Flutter dan Dart untuk membuat aplikasi mobile. Tahapan yang dikerjakan meliputi pengecekan dan konfigurasi Flutter, pembuatan project, pengenalan struktur project, serta penggunaan beberapa widget dasar seperti `MaterialApp`, `Scaffold`, `AppBar`, `Column`, `Text`, `Icon`, dan `SizedBox`. Selain itu, penggunaan `StatefulWidget` dan `setState()` juga membantu memahami bagaimana membuat tampilan aplikasi yang dapat berubah berdasarkan interaksi pengguna.